# Assignment 5: Multilayer Perceptron

Group A: Activation Function Comparison on MNIST

Anatoli Abahumna Leonardo Meloni

Class group 102, group 16, variant A

## 1 Introduction

The goal of this assignment is to implement a Multilayer Perceptron (MLP) from scratch using NumPy, train it on the MNIST image classification dataset, and analyze the effect of different hidden-layer activation functions.

The activation functions to be developed and evaluated are:

- **Tanh Function**
- **Sigmoid Function**
- **ReLU Function**
- **LeakyReLU** ($\alpha = 0.01$)

The performance of these functions will be systematically compared using an identical baseline architecture, optimizer (SGD), loss function (MSE), and hyperparameters. This approach isolates the effect of each activation, allowing us to identify the best-performing one. The optimal activation function will subsequently be tested on a secondary network architecture, followed by a critical discussion on its convergence speed and its susceptibility to the vanishing gradient problem.

## Activation Functions

The four hidden activation functions evaluated in this experiment, along with their respective properties, are detailed below. Their derivatives were implemented directly and used during backpropagation.

### 1. Tanh (Hyperbolic Tangent)

$$f(x) = (x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

A smooth, zero-centered curve mapping inputs to the $[-1, 1]$ range. While it centers data well, its derivative quickly approaches zero for large inputs (both positive and negative), making it highly susceptible to the vanishing gradient problem in deep architectures.

### 2. Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

An S-shaped curve that squashes values into the $[0, 1]$ range. Like Tanh, it suffers severely from vanishing gradients. Its maximum possible derivative is only 0.25, which drastically slows down the learning process when propagating errors backward through multiple layers.

### 3. ReLU (Rectified Linear Unit)

$$\text{ReLU}(x) = \max(0, x)$$

A piecewise linear function that outputs zero for negative inputs and the raw value for positive ones. Because its derivative for positive values is exactly 1.0, it effectively prevents the vanishing gradient problem, allowing the gradient to flow intact and significantly accelerating convergence.

### 4. LeakyReLU ($\alpha = 0.01$)

$$\text{LeakyReLU}(x) = \begin{cases} x, & x > 0 \\ 0.01x, & x \leq 0 \end{cases}$$

A variant of ReLU that introduces a slight mathematical slope for negative inputs instead of outputting a flat zero. This modification prevents the "dying ReLU" problem, ensuring that neurons receiving negative inputs can still calculate a small gradient and update their weights during training.

## 2 Experimental Setup

### 2.1 Network Architectures

All evaluated architectures share a fixed input layer of 784 neurons (corresponding to the flattened $28 \times 28$ pixel MNIST images) and an output layer of 10 neurons (representing the one-hot encoded probabilities for digits 0 through 9). To thoroughly investigate the network's learning capabilities, the hidden layers were configured into three distinct topologies. While all activation functions were initially tested on the baseline architecture, the wider and deeper configurations were evaluated exclusively using the best-performing activation function:

- **Baseline Architecture (784-64-32-10)**: This serves as the primary control configuration, utilizing a standard topology with two hidden layers. The progressive reduction in dimensionality forces the network to compress information and extract essential hierarchical features without incurring excessive computational costs.
- **Wider Architecture (784-128-64-10)**: This topology as the same depth as the baseline but significantly increases its width. The motivation is to test whether providing the network with a larger parameter capacity allows it to capture a wider variety of spatial patterns simultaneously, potentially improving accuracy.
- **Deeper Architecture (784-128-64-32-16-10)**: This configuration introduces four hidden layers, creating a noticeably deeper network. The motivation behind this design is to observe whether a deeper hierarchy of abstractions improves overall generalization.

### 2.2 Hyperparameters

To ensure a fair comparison and isolate the effect of each activation function, all initial tests were conducted using an identical baseline configuration. The choices for the hyperparameters were driven by the following motivations:

- **Data Split (Stratified 80/20)**: Ensures a consistent class distribution between the 56,000 training and 14,000 testing images, preventing statistical bias.
- **Reproducibility (Seeds 7, 21, 42)**: Averages out the variance caused by favorable or unfavorable random weight initializations, ensuring reliable and reproducible results.

- **Loss (MSE) and Learning Rate (0.1)**: A learning rate of 0.1 was chosen as a balanced "middle ground" for the MSE loss. It remains sufficiently high to prevent the complete stagnation of saturating functions, while ensuring a more stable and smoother convergence for ReLU and Tanh without the risk of overshooting the global minimum.
- **Optimizer (SGD) and Batch Size (128)**: Provides an optimal trade-off between computational efficiency through vectorization and the frequent weight updates needed to escape local minima.
- **Epochs (40)**: An extended training window of 40 epochs was selected to observe the complete learning trajectory. This duration is critical to reveal the "vanishing gradient" effect: it allows enough time for the slower Sigmoid function to eventually converge, while simultaneously monitoring if faster functions like ReLU maintain stability or begin to overfit in the later stages.

# 3 Accuracy Results - Training Loss Curves

Table 1 reports the mean test accuracy and standard deviation for each activation function over three random seeds, using a learning rate of 0.1 and trained for 40 epochs.

Table 1: Test accuracy by hidden activation over three seeds (LR = 0.1, Epochs = 40).

| Activation | Mean test accuracy | Standard deviation |
|---|---|---|
| Sigmoid | 0.9326 | 0.0006 |
| Tanh | 0.9713 | 0.0021 |
| ReLU | 0.9720 | 0.0008 |
| LeakyReLU | 0.9729 | 0.0003 |

### Discussion and Comparison

With proper weight initialization in place (Xavier), all models successfully converged, but their learning dynamics varied significantly. The results demonstrate a clear performance hierarchy driven by the mathematical properties of the activation functions.

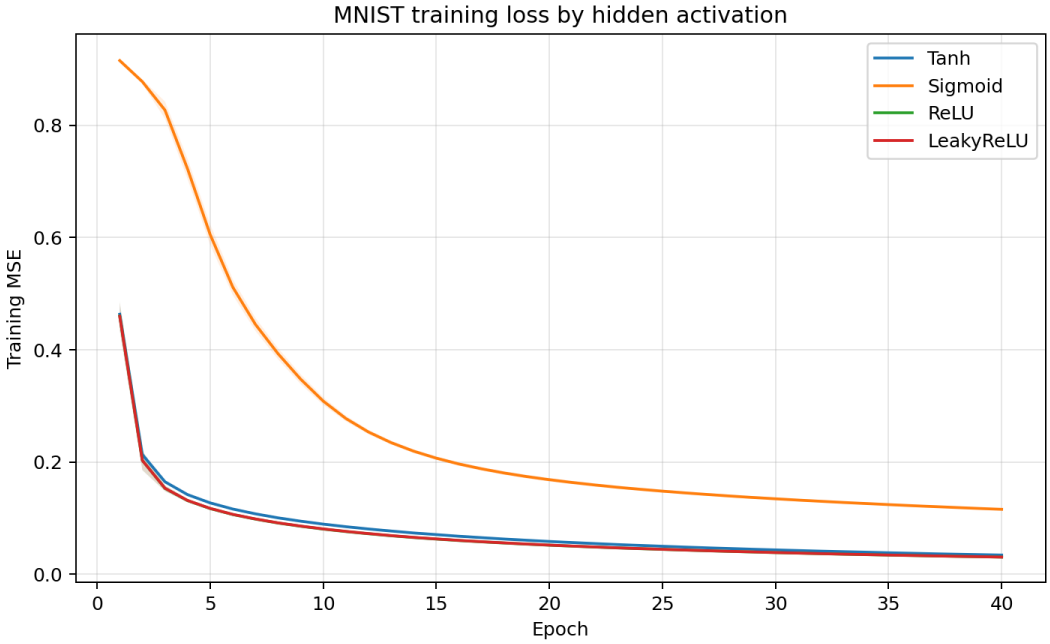



Figure 1: Training loss by hidden activation (the ReLU curve almost perfectly overlaps with LeakyReLU, rendering it barely visible).

- **Sigmoid's Slow Convergence**: It achieved the lowest accuracy (93.26%) and exhibited the slowest learning curve. This highlights the vanishing gradient problem, caused by its low maximum derivative (0.25), which severely limits weight updates.
- **High Efficiency of ReLU and LeakyReLU**: These were the top performers (achieving $\sim$ 97.2% accuracy), characterized by rapid error reduction within the first 5 epochs. LeakyReLU showed a marginal advantage in overall accuracy and stability.
- **Robustness of Tanh**: Despite being a saturating function, Tanh achieved a highly competitive accuracy of 97.13%. Its zero-centered property and steeper maximum derivative (1.0) successfully prevented the severe gradient decay observed in the Sigmoid function.

## 4 Architecture Comparison

Building upon the activation function results, the best-performing activation (LeakyReLU) was used to evaluate three different network architectures: the baseline (64-32), a wider network (128-64), and a deeper network. Table 2 summarizes the mean test accuracy and standard deviation across three random seeds.

Table 2: Test accuracy by architecture using LeakyReLU (LR = 0.1, Epochs = 40).

| Architecture | Mean test accuracy | Standard deviation |
|---|---|---|
| Baseline (64-32) | 0.9729 | 0.0003 |
| Wider (128-64) | 0.9761 | 0.0011 |
| Deep Architecture | 0.9749 | 0.0009 |

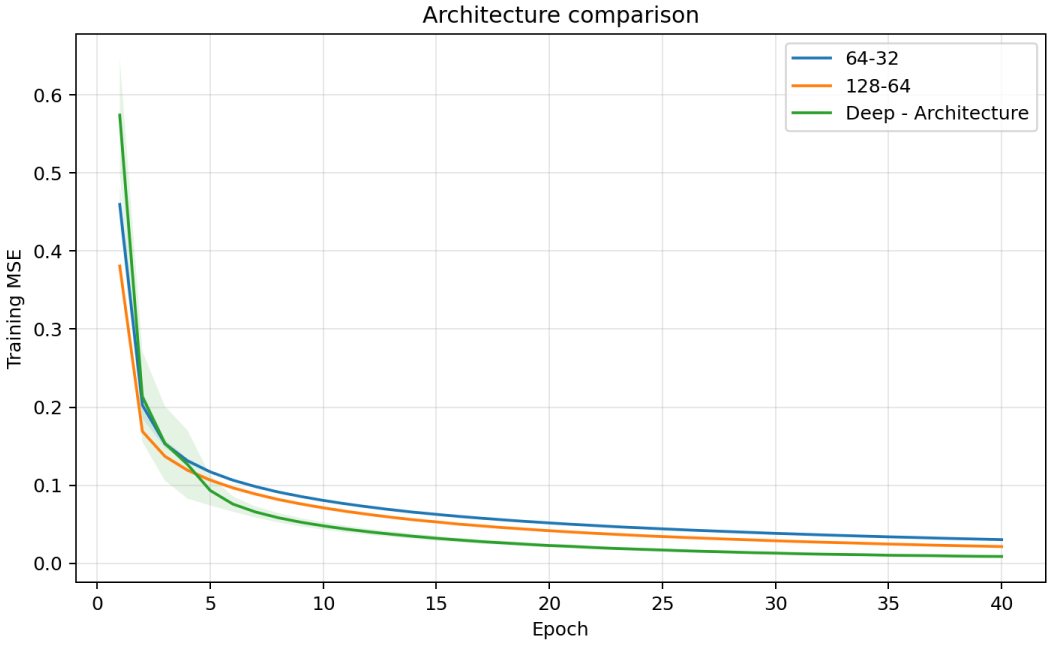



Figure 2: Mean training loss curves for different architectures using LeakyReLU.

## Discussion and Comparison

The comparison reveals important insights into model capacity and generalization. While all architectures achieved excellent performance, adding neurons or layers yielded different outcomes.

- **The Optimal Capacity (128-64)**: The "Wider" architecture proved to be the most effective, achieving the highest mean test accuracy of 97.61%. By doubling the number of neurons in the hidden layers compared to the baseline, the network gained sufficient capacity to learn more complex, fine-grained features from the dataset, translating to better generalization on unseen data.
- **Evidence of Overfitting (Deep Architecture)**: As observed in Figure 2, the deep architecture (green line) achieves the absolute lowest training MSE (dropping below 0.010). However, its test accuracy (97.49%) is lower than the wider, shallower network. This discrepancy indicates that the extra layers allowed the network to memorize the training data rather than extracting generalized patterns.
- **Baseline Stability (64-32)**: The baseline model, while having the lowest overall test accuracy (97.29%) and the highest final training loss, exhibited the greatest stability. Its extremely low standard deviation (0.0003) suggests that smaller networks are less sensitive to random weight initialization, though they ultimately lack the representational power to match wider architectures.

## The Impact of Learning Rate on Vanishing Gradients

To further investigate the relationship between activation functions and weight updates, an additional experiment was conducted comparing a lower learning rate ($\eta = 0.01$) with a higher learning rate ($\eta = 0.5$). The data, summarized in Table 3, reveals a critical interaction between the choice of hyperparameter and the vanishing gradient problem.

Table 3: Comparison of mean test accuracy across different learning rates.

| **Activation** | **Accuracy (LR = 0.01)** | **Accuracy (LR = 0.5)** |
|---|---|---|
| Sigmoid | 0.5988 | 0.9690 |
| Tanh | 0.9271 | 0.9738 |
| ReLU | 0.9330 | 0.9757 |
| LeakyReLU | 0.9323 | 0.9755 |

- **Sigmoid's Hyperparameter Dependency:** At a low learning rate ($\eta = 0.01$), Sigmoid fails catastrophically (59.88%) due to the vanishing gradient problem. An aggressive learning rate ($\eta = 0.5$) artificially rescues it (96.90%) by amplifying microscopic updates, proving its extreme sensitivity to exact hyperparameter tuning.
- **Robustness of ReLU and LeakyReLU:** ReLU and LeakyReLU demonstrate exceptional stability. Their non-saturating nature yields strong results ($> 93\%$) even at $\eta = 0.01$ and scales optimally ($\sim 97.5\%$) at $\eta = 0.5$, proving they do not rely on aggressive tuning to converge.
- **Tanh Resilience:** Tanh effectively bridges the gap. Its steeper maximum derivative prevents the complete collapse seen in Sigmoid at low learning rates (92.71%), while still leveraging the higher learning rate to achieve a highly competitive 97.38%.

# 5 Conclusions Summary

The experimental results highlight the critical impact of activation functions on learning dynamics:

- **Convergence Speed:** Rectified functions (ReLU and LeakyReLU) converge significantly faster than saturating ones, allowing for rapid error reduction within the initial epochs. However, Tanh still performed remarkably well; thanks to its zero-centered output and steeper maximum derivative, it achieved a highly competitive convergence rate, far outpacing the sluggish Sigmoid function.
- **Vanishing Gradients:** The slow convergence of Sigmoid perfectly illustrates the vanishing gradient problem, caused by its low maximum derivative (0.25) which severely shrinks weight updates. While Tanh mitigates this slightly, ReLU and LeakyReLU effectively bypass the issue by maintaining a constant gradient for positive activations.
- **Architecture and Generalization:** LeakyReLU yielded the highest mean accuracy by preventing "dead neurons" via its small non-zero gradient for negative inputs. Furthermore, pairing LeakyReLU with a wider architecture (128-64) produced the best overall performance, demonstrating that increased model capacity paired with a stable activation improves generalization without causing overfitting.